The Stata Journal (*yyyy*) *vv*, Number *ii*, pp. 1–28

wxsum: A command for processing temperature and precipitation data

Jeffrey D. Michler
University of Arizona
Tucson, AZ
jdmichler@arizona.edu

Anna Josephson
University of Arizona
Tucson, AZ
aljosephson@arizona.edu

Oscar Barriga-Cabanillas
World Bank
Washington, D.C.
obarriga@worldbank.org

Aleksandr Michuda
Swarthmore College
Swarthmore, PA
amichud1@swarthmore.edu

Jeffrey C. Oliver
University of Arizona
Tucson, AZ
jcoliver@arizona.edu

Abstract. Weather data are now a standard input in socioeconomic analysis, yet computing seasonal weather metrics from daily data remains error-prone and time-consuming. In this article, we introduce `wxsum`, a community-contributed command that reduces this cost. The command takes daily precipitation or temperature data and calculates 23 precipitation and 14 temperature measures, ranging from summary statistics (mean, standard deviation) to shock measures (z-scores, dry spells, killing degree days). By standardizing these calculations, `wxsum` lowers the probability of coding errors and allows researchers to move quickly from raw weather data to analysis-ready variables.

Keywords: st0001, wxsum, Earth observation, weather data, data management, summary statistics

1 Introduction

The expanding availability and decreasing cost of use have made weather data, particularly remotely sensed and/or gridded data, a common input into socioeconomic analysis (Gortan et al. 2024; Wuepper et al. 2025). Economists use these ready-made Earth observation products to predict a range of economic outcomes, including agricultural and industrial output, labor productivity, energy demand, health, conflict, human capital, and economic growth. Yet, there are many ways to quantify weather and very little theoretical or systematic guidance on which weather metric is best suited to a given research question and geographic context. The result is that seemingly similar measures of "rainfall," "temperature," or "shock" can be based on very different summaries of the underlying weather data (Michler et al. 2026a). The variety of metrics used and the size of weather data frames can create data-handling and cleaning problems, increasing the probability of coding errors when calculating any given metric.

In this article, we introduce the community-contributed command `wxsum`, which processes daily precipitation and temperature data and produces daily, monthly, seasonal, or annual summary statistics and shock measures. The goal of the command is to enable easy, error-free calculation of the most common weather metrics used in the

© *yyyy* StataCorp LLC

socioeconomic literature. The `wxsum` command takes in daily precipitation or temperature data, typically in the form of gridded Earth observation data, and calculates a number of summary statistics for both, such as the mean, median, variance, and skew, as well as phenomenon-specific statistics, such as total rainfall, maximum temperature, and growing degree days (GDD), and several measures of shocks, including deviations from long-run averages, z-scores, dry spells, and heat spells.

The `wxsum` command allows the user to customize the dates and lengths of "seasonal" windows, the rainfall threshold defining a rainy day, GDD bounds, extreme heat thresholds, daily-temperature bins, the length of the long-run reference window, and the final output shape (long or wide). This flexibility is important because the relevant exposure period, temperature sensitivity, and baseline climate vary across research questions. For example, a study of crop yields might measure rainfall during the growing season, whereas a study of birth outcomes might measure heat exposure over the nine months preceding birth. The command accommodates these differences. Additionally, a companion command, `wxprep`, reshapes and pre-processes the data for the user into the needed format for use in `wxsum`.

The choice of metric should follow from the research question and the mechanism under study (Michler et al. 2026b). For agricultural applications, total seasonal rainfall, rainy-day counts, and the lengths of early, middle, and late dry spells capture different dimensions of water availability, particularly when the timing and persistence of rainfall deficits matter for crop yields (Marcos-Garcia et al. 2024). Growing degree days, extreme-heat days (killing degree days, KDD), and temperature bins capture the accumulated heat exposure relevant to plant development and damage (Schlenker and Roberts 2009).

Beyond agriculture, rainfall deviations from long-run averages and corresponding z-scores capture weather shocks relative to local climatic conditions which can be relevant for household consumption and poverty (Azzarri and Signorelli 2020) as well as social conflict (Hendrix and Salehyan 2012). Temperature bins, extreme-heat, and heat spells capture heat exposure which can be relevant for prenatal development, (Doyle and Leckning 2025), student learning (Park et al. 2020), labor productivity(Somanathan et al. 2021), and residential electricity demand (Auffhammer and Aroonruengsawat 2011). These are not exclusive categories. Many applications combine several metrics to capture different dimensions of weather exposure. Additionally, we have developed an implementation for R (`wxsumR`), extending the same core functionality to users who integrate weather processing into R-based data pipelines.

While the preceding guidance reflects common practice in the literature, the command itself is agnostic about which of the many weather metrics a researcher uses. In creating the `wxsum` command, we are not endorsing an atheoretical or ad hoc approach to incorporating weather measures into socioeconomic analysis. A growing body of literature raises concerns about the use (and misuse) of weather data, particularly when weather is used to identify exogenous variation (Benami et al. 2026). These concerns include how gridded weather data is matched with socioeconomic data (Michler et al. 2022), measurement error in Earth observation products (Proctor et al. 2023; Joseph-

son et al. 2026), and the validity of the assumption that weather is truly exogenous (Sarsons 2015; Mellon 2025). While incorporating weather data into one's analysis is now relatively low cost, a naïve or unexamined use of that data will continue to carry a high cost in terms of the accuracy of one's analysis and results.

2 Data considerations

The `wxsum` command is designed for a common structure of remotely sensed or gridded Earth observation products after they have been matched to locations in an analysis dataset. Each observation (row) is a location, plot, household, firm, or other spatial unit, and each variable (column) is a daily measurement for that unit. The command assumes this spatial matching step has been completed. Before running `wxsum`, the user should decide which weather product, temporal aggregation, and spatial linking rule are appropriate for the research design. These choices are substantive, not merely computational, as estimates can be sensitive to the source of weather data and to the way gridded observations are linked to socioeconomic observations (Auffhammer et al. 2013; Michler et al. 2026b).

The `wxsum` command requires Stata 15 or later and for the data to be in wide format. A dataset with 40 years of daily observations implies roughly 14,600 variables per observation. Stata/BE limits datasets to 2,048 variables, which restricts the command to approximately five years of daily data. Stata/SE supports up to 32,767 variables, and Stata/MP supports up to 120,000. Users working with long time series in Stata/BE may need to split the data into shorter panels, run `wxsum` on each, and append the results.[1]

2.1 Shape and naming convention

Daily weather values are stored in separate numeric variables whose names share a common prefix (stub) and end with an eight-digit date in `yyyymmdd` format. For example, if the precipitation variables are named with the prefix `rf_`, then precipitation on May 15, 1979, should appear in a variable named `rf_19790515`. A similar approach applies for temperature variable names after the prefix. Variables not beginning with the specified stub are ignored unless they are retained using the `keep()` option.

The companion command `wxprep` prepares weather data that do not follow this structure. It converts long-format data to the required wide format, renames existing daily variables to follow the date-based naming convention, and aggregates sub-daily readings into daily summaries. It can also convert wide-format data back to long form when users need to perform additional transformations before calculating weather metrics. Data that already satisfy the input requirements can be passed directly to `wxsum`.

The wide format, while memory-intensive, is a natural representation of the grid-

1. Splitting the data affects the reference period used for calculating deviations and z-scores. Users who split their data should ensure sufficient overlap to avoid truncating the reference period.

ded Earth observation products common in the climate science literature. The `wxsum` command is agnostic to the source of the data. It can process data from any product that provides daily or subdaily readings on a regular grid. Widely used products of this type include CHIRPS (Funk et al. 2015), CPC (Chen et al. 2008), ARC2 (Novella and Thiaw 2013), TAMSAT (Tarnavsky et al. 2014), ERA5 (Hersbach et al. 2018), and MERRA-2 (Bosilovich et al. 2016). It can also process data from ground-based weather station networks.[2]

The required naming convention for the input data makes the calendar explicit. The command parses the date suffix of each weather variable, identifies the days that belong to each user-defined season or window, and computes statistics separately for each location and each window. As all computations are based on the dates in variable names, users should check that the date suffixes are complete, unique, and measured consistently on the intended calendar and time zone. The command can accommodate windows that cross calendar years. For example, when a window spans two calendar years, it is assigned to the year in which it begins. A window running from November 2001 to February 2002 is labeled 2001 in the command's output.

2.2 Windows, units, and missing values

Users can define the relevant time window in one of three ways. The default is to define a calendar window with an initial month and final month and, optionally, an initial day and final day. This fixes the window to the same start and end date for all units and for all years. Alternatively, the user can define unit-specific windows. These can be defined by providing the command with a variable that holds each observations' start and end date, such as a planting date and harvest date. Or, the window can be defined with a variable that holds an event date for each observations as well as the number of days the event lasted, such as a birth date and the length of the gestation period. In both cases, the exposure window is defined separately for each observation rather than being fixed to the same calendar dates across observations and years. The command does not impose a biological or econometric interpretation on the chosen window. Rather, the user must define the exposure window before running the command and should report that choice in subsequent empirical work.

The command is similarly agnostic about units. Rainfall can be measured in millimeters, centimeters, or inches, and temperature can be measured in degrees Celsius, Fahrenheit, or Kelvin. It is necessary, however, that all daily variables are measured consistently with the same unit used throughout. Unit choices matter for thresholds. For example, the lower and upper bounds for GDD and the base for KDD must be expressed in the same units as the daily temperature data. Similarly, the rainy-day threshold must be expressed in the same units as the daily precipitation data.

2. While designed for temperature and precipitation data, the command's agnosticism means one could pass it any type of daily weather data, such as humidity, wind speed, wet bulb temperature, or standardized precipitation evapotranspiration index (SPEI). However, not every metric that `wxsum` produces will have a meaningful interpretation when the data is not temperature or precipitation.

Many weather products contain missing values due to sensor problems, limited spatial coverage, cloud contamination, or limitations in station interpolation (Dinku 2020). Users should examine missingness in the daily variables and decide whether the weather product and temporal coverage are adequate for the application. For means, medians, standard deviations, maxima, rainy-day counts, rain-day shares, daily-temperature bin counts, and dry spells and heat spells, missing daily values are not treated as observed weather. For seasonal and monthly precipitation totals, missing daily values contribute zero to those sums. This means seasons with missing days will understate total rainfall. The command has an option that allows the user to define the minimum number of non-missing observations that are acceptable in a window. It also produces stored results that capture the degree of missingness in the unprocessed weather data.

3 The `wxsum` and `wxprep` commands

The `wxsum` command calculates weather statistics for calendar or unit-specific windows. The companion command `wxprep` prepares the daily data by reshaping observations, renaming variables, or aggregating multiple readings within a day. This section describes the syntax, options, output, and stored results of each command.

3.1 The `wxsum` command

Syntax

For a calendar window, the same starting and ending dates are applied to every observation, with one window per year:

```
wxsum stub, type(rain|temp) ini_month(#) fin_month(#) [ ini_day(#)
    fin_day(#) options ]
```

For a unit-specific window, the starting and ending dates are read from variables, with one window per observation:

```
wxsum stub, type(rain|temp) start(varname) end(varname) [ partial options
    ]
```

Alternatively, a unit-specific window can be defined relative to an event date:

```
wxsum stub, type(rain|temp) event_date(varname) days(#) [ lag(#) partial
    options ]
```

Here *stub* is the common prefix for the daily weather variables. It is not a varlist. If the data contain variables named `rf_19790101`, `rf_19790102`, and so on, the stub supplied to the command is `rf_`. The user specifies `type(rain)` or `type(temp)` and

one of the three window definitions. With `type(temp)`, `gdd_lo()`, `gdd_hi()`, and `heat_threshold()` are also required. The remaining options are described below.

Required variables

Input data must be wide, with one row per observation and separate numeric variables for daily weather values. Each daily variable name consists of the supplied stub followed by a date in `yyyymmdd` format. A calendar day without a corresponding variable is treated as missing. Unit-specific windows additionally require variables containing Stata daily dates. Identifiers are not arguments of `wxsum`, but can be retained with `keep()` for linking the results to other data. Table 1 summarizes the input requirements.

Table 1: Data requirements for `wxsum`

Element	Requirement
Observation	One row per spatial or analytic unit.
Weather variables	Numeric daily values named with a common stub and a `yyyymmdd` suffix.
Calendar window	Starting and ending months, and optional days, specified in the command.
Unit-specific window	Stata daily-date variables for the start and end dates, or an event-date variable and a specified number of preceding days.
Data type	`type(rain)` for precipitation or `type(temp)` for temperature.
Other variables	Retain with `keep()` if identifiers, coordinates, or covariates should remain in the output.

Options

Calendar windows

`ini_month(#)` and `fin_month(#)` specify the first and last month of a calendar window, from 1 through 12. Both are required for this window definition. When the final month precedes the initial month, the window crosses the end of the calendar year and is associated with the year in which it starts.

`ini_day(#)` and `fin_day(#)` specify the first and last day within those months. By default, the window begins on the first day of the initial month and ends on the last day of the final month, including February 29 in leap years. If February 29 is explicitly specified, February 28 is used in common years.

Unit-specific windows

`start(varname)` and `end(varname)` identify variables containing the first and last day of each observation's window as Stata daily dates (`%td`). Windows can differ in

length for each observation. An observation with a missing date receives missing statistics. Datetimes in `%tc` format are not accepted directly; they must first be converted to daily dates using `dofc()` or the `convert()` option of `wxprep`.

`event_date(varname)` **and** `days(#)` define a window of `#` days ending the day before an event. Windows have a common length but the event-date variable allows the dates to differ across observations.

`lag(#)` shifts an event-based window by the specified number of days. The default is 0. A positive value moves the window earlier and a negative value moves it later. The resulting start and end dates are `event_date - days - lag` and `event_date - 1 - lag`, respectively.

`partial` uses the available days when the window extends before the first or after the last date of the daily variables for a given unit of observation. Without this option, such windows receive missing statistics. This option does not change the calendar-window requirement that the entire window lie within the data's date range.

With either unit-specific definition, the output contains one row per input observation and generated variable names have no year suffix. Because each observation has only one exposure window, the `shape()` option does not apply.

Rainfall and temperature thresholds

`rain_threshold(#)` defines a rainy day as one with precipitation at or above the threshold and a dry day as one with precipitation below it. The default is 1. This option applies only to `type(rain)`.

`gdd_lo(#)` **and** `gdd_hi(#)` specify the lower and upper bounds for GDD. Both are required with `type(temp)`.

`heat_threshold(#)` defines a heat day as one with temperature at or above the threshold. It is required with `type(temp)` and is used to calculate heat-day counts and heat spells. It also sets the KDD base unless `kdd_base()` is specified. The interpretation depends on the supplied daily series: a threshold applied to daily mean temperature measures a different exposure from the same threshold applied to daily maximum temperature.

`kdd_base(#)` optionally specifies a separate base for KDD. Without this option, the heat threshold is also the KDD base.

All thresholds must use the units of the daily data. The command does not convert units.

Optional bins and categories

`gdd_bin(#)` specifies the width (in GDD units) of GDD bins. When specified, the command creates `gddcat`, an integer category identifying the fixed-width interval containing the window's GDD total. Categories are pooled across observations and windows and have value labels identifying the intervals, so `i.gddcat` can be used in regressions. This option applies only to `type(temp)`.

`gdd_binlo(#)` specifies the lower endpoint of the regular GDD intervals when `gdd_bin()` is used. The default is 0. Totals below this endpoint receive a bottom-coded category.

`gdd_binhi(#)` specifies the upper endpoint of the regular GDD intervals. Totals at or above it receive a top-coded category. When it is omitted, the intervals extend beyond the largest GDD total. The difference between the upper and lower endpoints must be a multiple of the width specified in `gdd_bin()`.

`tmp_bin(#)` specifies the number of daily temperature bins, from 1 through 42. It requires `tmp_binlo()` and `tmp_binhi()`. For three or more bins, the first counts days below the lower endpoint, the last counts days at or above the upper endpoint, and the remaining bins divide the interval between the endpoints into equal widths. Interior bins include their lower boundary and exclude their upper boundary. `tmp_bin(1)` counts all observed days; `tmp_bin(2)` divides them at the midpoint of the two endpoints. The generated variables are named `tmpbin01`, `tmpbin02`, and so on.

`tmp_binlo(#)` **and** `tmp_binhi(#)` specify the lower and upper endpoints. These are absolute temperature thresholds, not percentiles of the observed distribution.

`cuts(numlist)` creates daily bin counts using up to 41 increasing cutpoints. The first variable, `cut01`, counts days below the first cutpoint. Subsequent variables count days from one cutpoint up to, but not including, the next. The final variable counts days at or above the last cutpoint. Thus K cutpoints create $K + 1$ bins. This option can be used with either data type, for example to define precipitation-intensity or temperature-exposure bands. Cannot be combined with `tmp_bin()`.

Daily bins count observed days rather than shares of days. Their values are missing when a window has fewer observed days than required by `minobs()`. Output names receive a year suffix only for calendar windows with wide output, as described below.

Reference periods

`lr_years(#)` specifies a rolling reference period consisting of windows starting in the indicated number of preceding years. The permitted range is 2 through 50 and the default is 10. The current window is excluded from its own rolling reference period.

`baseline(# #)` specifies a fixed reference period. For example, `baseline(1981 2010)` uses windows starting in 1981 through 2010. Windows inside the baseline are

compared with a baseline that includes them. The baseline must cover at least `ref_min()` windows of the data, or years of the data for unit-specific windows. Cannot be combined with `lr_years()`.

`ref_min(#)` specifies the minimum number of nonmissing reference values an observation needs for a deviation or z-score. It must be at least 2. By default, all reference years are required: the number specified in `lr_years()` or the number of years in `baseline()`.

For unit-specific windows, the reference values are calculated for the same window moved back to each preceding year or to each baseline year. February 29 is moved to February 28 in common years. The reference period and `ref_min()` govern deviations and z-scores rather than the nonreference summaries of the current window.

Missing data

`minobs(#)` specifies the minimum number of nonmissing daily observations required to calculate statistics for a window (default: 1). If this requirement is not met, all weather statistics are set to missing.

Output

`shape(long|wide)` specifies the output shape for calendar windows. The default, `long`, produces one row per observation and window, a variable named `year` identifying the window's starting year, and statistic names without year suffixes. `wide` produces one row per observation and statistic names with year suffixes. Input data must be wide in either case. This option has no effect on unit-specific windows.

`outstub(string)` adds a prefix to every generated variable name except `year`. For example, `outstub(r_)` gives `r_total`. Separate output prefixes allow rainfall and temperature results, or results for different windows, to be kept together.

`keep(varlist)` specifies which variables from the input dataset are retained in the output. For calendar windows with `shape(long)`, the output contains only the generated statistics and `year` by default. The `keep()` option allows users to retain additional variables, such as identifiers. For calendar windows with `shape(wide)` and for unit-specific windows, all input variables are retained by default. Specifying `keep()` restricts the output to the listed input variables and the generated statistics.

`save(filename [, replace])` saves the output dataset and leaves the results in memory. To overwrite an existing file, specify `replace` within the option, as in `save(rain_out, replace)`.

Output

`wxsum` is a data-management command that produces a dataset of window-level weather statistics. For calendar windows, the default `shape(long)` output contains one row per observation and year, with a variable named `year` identifying the window. With `shape(wide)`, each input observation remains one row and the generated statistics receive year suffixes, such as `mean_1993` and `total_1993`. Windows crossing calendar years are assigned to the year in which they begin. For unit-specific windows, the output contains one row per input observation, and generated statistics have no year suffix. Tables 2 and 3 list rainfall and temperature statistics without an optional output prefix or year suffix.

For rainfall, `wxsum` generates measures of the distribution, amount, frequency, and persistence of precipitation. In addition to standard distributional statistics, the command calculates mean monthly precipitation and total rainfall over the exposure window. Rainy-day counts and shares capture the frequency of precipitation, while dry-spell measures distinguish the duration of rainfall deficits at the beginning, within, and at the end of the window. These measures capture dimensions of rainfall exposure that may not be apparent from total precipitation alone. Optional bins further characterize the distribution of daily rainfall intensity.

Table 2: Rainfall statistics generated by `wxsum`

Group	Generated statistics
Observed days	`nobs`: number of nonmissing daily observations.
Daily distribution	`mean`, `median`, `var`, `sd`, and `skew`.
Monthly rainfall	`mean_mo`: mean of monthly precipitation totals.
Total rainfall	`total`: total precipitation during the window.
Rainy and dry days	`raindays` and `norain`: counts of rainy and non-rainy days; `pct_raindays`: share of observed days with rain.
Dry spells	`dry_start`, `dry`, and `dry_end`: lengths of leading, mid-window, and trailing dry spells.
Reference measures	Deviations (`dev_`) and z-scores (`z_`) for `mean_mo`, `total`, `raindays`, `norain`, `pct_raindays`, and `dry`.
Optional bins	`cut01`, `cut02`, etc.: counts of days in user-defined rainfall intervals.

For temperature, `wxsum` generates measures of the daily temperature distribution, accumulated thermal exposure, and the frequency and duration of extreme heat. Growing and killing degree days capture cumulative exposure relative to specified temperature thresholds, whereas heat-day counts and shares measure the frequency of threshold exceedance. Heat-spell measures distinguish persistent heat exposure at the beginning, within, and at the end of the window. Optional temperature bins capture the distribution of daily temperatures across specified intervals, while GDD categories classify accumulated growing degree days.

Table 3: Temperature statistics generated by `wxsum`

Group	Generated statistics
Observed days	`nobs`: number of nonmissing daily observations.
Daily distribution	`mean`, `median`, `var`, `sd`, `skew`, and `max`.
Degree days	`gdd`: accumulated growing degree days; `kdd`: accumulated killing degree days.
Heat days	`heatdays`: number of days meeting the heat threshold; `pct_heatdays`: share of observed days meeting the threshold.
Heat spells	`heat_start`, `heat`, and `heat_end`: lengths of leading, mid-window, and trailing heat spells.
Reference measures	Deviations (`dev_`) and z-scores (`z_`) for `gdd`, `kdd`, `heatdays`, `pct_heatdays`, and `heat`.
Optional GDD categories	`gddcat`: categorical measure of accumulated GDD, with value labels identifying the intervals.
Optional daily bins	`tmpbin01`, `tmpbin02`, etc., or `cut01`, `cut02`, etc.: counts of days in temperature intervals.

For both rainfall and temperature, the command calculates deviations and z-scores for selected statistics relative to a rolling or fixed reference period. These measures distinguish weather exposure from conditions that are unusual relative to the historical experience of a location. The variables `pct_raindays` and `pct_heatdays` report shares, not percentages. Both outputs include `nobs`, the number of nonmissing daily observations, to help assess data completeness.

Finally, the generated variables include labels identifying the statistics and relevant thresholds. Variable and dataset characteristics record the exposure windows and command settings, providing a record of how the weather measures were constructed that persists when the output dataset is saved.

Stored results

The `wxsum` command stores results in `r()`. The scalar and macro results are listed in Tables 4 and 5. Results specific to a data type, window definition, or reference mode describe that mode. In particular, `r(N_units)` counts distinct combinations of the variables in `keep()`, rather than relying on a separately specified unit identifier.

The matrix `r(missing)` summarizes daily-data coverage. For calendar windows, it has one row per window and columns `days`, the number of calendar days; `mean_nobs`, the mean number of observed days; `any_missing`, the share of observations with at least one missing day; `all_missing`, the share with no observed days; and `below_minobs`, the number below the required observation count. For unit-specific windows, a single summary row also reports `outside`, the number of windows extending beyond the data's date range, and `missing_dates`, the number with missing window dates. The

Table 4: Scalars stored by `wxsum`

Result	Description
`r(N)`	Number of observations in the output.
`r(N_units)`	Number of unique combinations of variables in `keep()`.
`r(k_created)`	Number of variables created.
`r(n_windows)`	Number of calendar windows; 1 for unit-specific windows.
`r(first_year)`	First calendar-window starting year.
`r(last_year)`	Last calendar-window starting year.
`r(minobs)`	Minimum required observed days per window.
`r(ref_min)`	Minimum required nonmissing reference values.
`r(lr_years)`	Reference-window length for rolling references.
`r(baseline_first)`	First year of a fixed baseline.
`r(baseline_last)`	Last year of a fixed baseline.
`r(rain_threshold)`	Rainy-day threshold for rainfall data.
`r(gdd_lo)`, `r(gdd_hi)`	GDD bounds for temperature data.
`r(heat_threshold)`	Heat threshold for temperature data.
`r(kdd_base)`	KDD base used for temperature data.

Table 5: Macros stored by `wxsum`

Result	Description
`r(cmdline)`	Command as typed.
`r(type)`	`rain` or `temp`.
`r(stub)`	Common prefix of the daily variables.
`r(outstub)`	Output-variable prefix.
`r(mode)`	`calendar` or `unit`.
`r(shape)`	`long`, `wide`, or `unit`.
`r(window)`	Window description, for example `15may-15oct`.
`r(years)`	Starting years of calendar windows.
`r(ref)`	Reference period, for example `rolling 10` or `baseline 1981-2010`.
`r(varlist)`	Names of the created variables.

`outside` windows receive missing statistics unless `partial` is specified. The matrix can be inspected with `matrix list r(missing)`.

3.2 The `wxprep` command

The `wxprep` command prepares the daily data required by `wxsum`. It has three operations: `towide` converts long-format observations to daily wide-format variables; `rename` changes existing wide-format variable names to the required date-based convention; and `tolong` converts daily wide-format variables back to long observations. The last operation allows users to carry out transformations, such as unit conversions or combining daily series, before returning to wide form.

Syntax

To convert long data, with one row per unit-day or per unit-reading, to wide data, the syntax is

```
wxprep towide varname [ if exp ] [ in range ], id(varlist) date(varname)
    stub(string) [ daily(mean|sum|min|max) dateformat(string) nofill
    convert(varlist) clear save(filename [, replace]) ]
```

The first *varname* identifies the numeric weather series. `id()` identifies the units, `date()` identifies the date variable, and `stub()` supplies the prefix for the new daily variables. The `if` and `in` qualifiers can restrict which observations are converted.

For wide data whose daily variables represent consecutive dates in a specified order, the syntax is

```
wxprep rename varlist, stub(string) firstdate(date) [ convert(varlist) ]
```

For wide data whose variable names contain dates after an old prefix, the syntax is

```
wxprep rename varlist, stub(string) from(string) dateformat(string) [
    convert(varlist) ]
```

To convert daily wide-format variables to one row per unit-day, the syntax is

```
wxprep tolong stub, id(varlist) [ generate(newvar) date(newvar)
    convert(varlist) clear save(filename [, replace]) ]
```

In `tolong`, *stub* identifies the existing daily weather variables. The meanings of `date()`, `id()`, and `convert()` depend on the operation, as described below.

Options

Options for `wxprep towide`

`id(varlist)` identifies the units and is required. The identifier variables can be numeric or string, and each distinct combination becomes one output row. Missing identifiers are not allowed. Apart from the daily variables it creates, `towide` retains only the `id()` variables.

`date(varname)` identifies the date of each reading and is required. It accepts Stata daily dates (`%td`), datetimes (`%tc` or `%tC`, retaining the day), integers in `yyyymmdd` form, or string dates accompanied by `dateformat()`.

`stub(string)` specifies the prefix of the new daily variables and is required. It is used exactly as typed and can contain at most 24 characters. For example, `stub(rf_)` produces names such as `rf_19930515`.

`daily(mean|sum|min|max)` combines multiple readings for the same unit and day using their mean, sum, minimum, or maximum. Examples include `daily(mean)` for daily mean temperature, `daily(sum)` for accumulated daily precipitation, and `daily(max)` for daily maximum temperature. Missing readings are ignored. Without `daily()`, duplicate unit-day readings produce an error.

`dateformat(string)` gives the order of year, month, and day in a string date. For example, the mask `YMD` reads `2020-01-31`, whereas `DMY` reads `31jan2020`.

`nofill` creates variables only for dates appearing in the data. By default, `towide` creates a variable for every calendar day between the first and last date, leaving days without readings missing. These are missing days, not zero weather values.

`convert(varlist)` converts additional datetime variables to daily dates by removing the time of day. Variable names and labels are preserved. The listed variables must also appear in `id()` so they are retained in the output. The weather-reading datetime supplied in `date()` is converted automatically and does not need to be listed here. The converted dates can then define unit-specific exposure windows in `wxsum`. Conversion occurs before grouping, so records distinguished only by different times on the same day will be treated as belonging to the same unit.

`clear` permits replacement of data in memory that have changed since they were last saved.

`save(filename [, replace])` saves the wide-format output; `replace` permits overwriting an existing file.

Options for `wxprep rename`

`stub(string)` specifies the prefix of the new names and is required.

`firstdate(date)` specifies the date of the first variable when the listed variables represent consecutive days in the order supplied. Accepted representations include `01jan1993`, `1993-01-01`, and `19930101`.

`from(string)` **and** `dateformat(string)` are used instead of `firstdate()` when each existing name contains its date. `from()` identifies the old prefix and `dateformat()` specifies the order of the date components that follow it. For names such as `rain_01jan1993`, use the old prefix `rain_` with the mask `DMY`.

`convert(varlist)` converts `%tc` or `%tC` datetimes to `%td` daily dates, preserving names and labels. With `rename`, these can be any variables other than the daily weather variables being renamed.

Before making changes, `rename` checks that the dates are valid, no two variables refer to the same date, and the new variable names do not already exist. It reports gaps between the first and last date.

Options for ***wxprep tolong***

`id(varlist)` is required and must uniquely identify the input observations. Variables other than the identifiers and the daily weather series are dropped. Unit-level variables that should be retained must therefore be included in `id()`.

`generate(newvar)` names the new variable holding the daily weather values. The default is the stub with a trailing underscore removed; for example, `rf_` gives `rf`. The option can be abbreviated to `gen()`.

`date(newvar)` names the new Stata daily-date variable. The default name is `date`, with format `%td`. Unlike `date()` in `towide`, this option names an output variable rather than identifying an input variable.

`convert(varlist)` converts `%tc` or `%tC` datetime variables to `%td` daily dates, preserving names and labels. The variables must appear in `id()`, and the identifiers must remain unique after conversion.

`clear` permits replacement of data in memory that have changed since they were last saved.

`save(filename [, replace])` saves the long-format output; `replace` permits overwriting an existing file.

Output and stored results

The `towide` operation produces one row per combination of identifier values and one numeric variable per daily date. The `rename` operation preserves the wide layout while changing the selected weather-variable names to the required stub and date suffix. The

Table 6: Results stored by `wxprep`

Result	Description
Panel A: `wxprep towide`	
`r(N)`	Number of units (output rows).
`r(n_days)`	Number of daily variables created.
`r(n_filled)`	Number of days created without data.
`r(n_combined)`	Number of readings combined using `daily()`.
`r(first_date)`, `r(last_date)`	First and last date.
`r(stub)`	Prefix of the new daily variables.
`r(first_var)`, `r(last_var)`	First and last new variable names.
`r(converted)`	Variables converted to daily dates.
Panel B: `wxprep rename`	
`r(n_vars)`	Number of variables renamed.
`r(n_gaps)`	Number of dates without a daily variable between the first and last date.
`r(first_date)`, `r(last_date)`	First and last date.
`r(stub)`	Prefix of the renamed daily variables.
`r(first_var)`, `r(last_var)`	First and last new variable names.
`r(converted)`	Variables converted to daily dates.
Panel C: `wxprep tolong`	
`r(N)`	Number of output observations (units multiplied by days).
`r(n_units)`, `r(n_days)`	Number of units and days.
`r(first_date)`, `r(last_date)`	First and last date.
`r(generate)`, `r(date)`	Names of the new weather and date variables.
`r(converted)`	Variables converted to daily dates.

`tolong` operation produces one row per unit-day with identifier variables, a date variable, and a weather-value variable. The date-filling rule for `towide` is controlled by `nofill`, and `towide` checks `maxvar` before reshaping. Stored results depend on the operation. Table 6 lists the results for `towide`, `rename`, and `tolong`.

4 Formulas and Calculations

This section defines the variables generated by `wxsum`. Let i index an input observation and let t denote the year in which its exposure window begins. Let $\mathcal{W}_{it}$ denote the window of observation i that starts in year t: the set of consecutive dates from the window's first date through its last date. For calendar windows, these dates are common across observations in a given year. For unit-specific windows, they are read from the starting and ending date variables or determined from the event date, window length, and lag. The same notation and calculations therefore apply to all three window definitions.

Let R_{id} denote daily precipitation on date d and T_{id} daily temperature. Write X_{id} for the series being processed, either R_{id} or T_{id}. A day of the window is observed when the data contain a daily variable for that date and the variable is not missing for observation i. Days with missing values, dates without a daily variable, and, for a unit-specific window with `partial`, dates outside the date range of the data are all missing days. Let $\mathcal{O}_{it} \subseteq \mathcal{W}_{it}$ be the set of observed days of the window. Their number, N_{it}, is reported as `nobs`:

$$\texttt{nobs}_{it} = N_{it} = |\mathcal{O}_{it}| \,. \tag{1}$$

Every other statistic is computed from the observed days alone and is missing when N_{it} is smaller than the value supplied in `minobs()`. With the default, `minobs(1)`, this applies only to windows without any observed day. The formulas below apply to windows that the command evaluates.

4.1 Statistics common to rainfall and temperature

For either rainfall or temperature data, the daily sample mean is

$$\texttt{mean}_{it} = \frac{1}{N_{it}} \sum_{d \in \mathcal{O}_{it}} X_{id}. \tag{2}$$

The daily median, $\texttt{median}_{it}$, is the median of the nonmissing values $\{X_{id} : d \in \mathcal{O}_{it}\}$, averaging the two central values when N_{it} is even. The daily sample variance requires at least 2 observed days and is

$$\texttt{var}_{it} = \frac{1}{N_{it} - 1} \sum_{d \in \mathcal{O}_{it}} \left(X_{id} - \texttt{mean}_{it} \right)^2 . \tag{3}$$

The daily standard deviation, $\texttt{sd}_{it}$, is the square root of the variance. The skewness variable is the adjusted Fisher–Pearson sample skewness,

$$\texttt{skew}_{it} = \frac{N_{it}}{(N_{it} - 1)(N_{it} - 2)} \sum_{d \in \mathcal{O}_{it}} \left(\frac{X_{id} - \texttt{mean}_{it}}{\texttt{sd}_{it}} \right)^3 . \tag{4}$$

Skewness is missing when fewer than three days are observed or the standard deviation is zero.

4.2 Rainfall calculations

For precipitation data, total rainfall over the window is

$$\texttt{total}_{it} = \sum_{d \in \mathcal{O}_{it}} R_{id}. \tag{5}$$

Missing days add nothing to the sum, so a window with missing days can understate the total; `nobs` shows how many days the total covers.

For monthly totals, the observed days are grouped by calendar month, where a month m is the month of a particular year and let $m(d)$ identify the year–month containing date d. Let $\mathcal{M}_{it}$ be the set of months that contain at least one observed day of the window. The total of month $m \in \mathcal{M}_{it}$ is

$$M_{imt} = \sum_{\substack{d \in \mathcal{O}_{it} \\ m(d) = m}} R_{id}. \tag{6}$$

and the mean of these monthly totals is

$$\texttt{mean_mo}_{it} = \frac{1}{|\mathcal{M}_{it}|} \sum_{m \in \mathcal{M}_{it}} M_{imt}. \tag{7}$$

A month that the window covers only in part, at its start or end, counts as a month, and a month without an observed day is left out rather than counted as a month without rain.

Let c denote the value supplied in `rain_threshold()`. Rainy days and days without rain are counted only among observed daily rainfall values while the variable `pct_raindays` is the share of observed days with rain:

$$\texttt{raindays}_{it} = \sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{R_{id} \geq c\}, \tag{8}$$

$$\texttt{norain}_{it} = \sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{R_{id} < c\}, \tag{9}$$

$$\texttt{pct_raindays}_{it} = \frac{\texttt{raindays}_{it}}{N_{it}}. \tag{10}$$

Here $\mathbf{1}\{\cdot\}$ is an indicator equal to one when its condition holds and zero otherwise. The dry spells are defined in section 4.4.

4.3 Temperature calculations

For temperature data, let h denote the value supplied in `heat_threshold()`. An observed day is a heat day when $T_{id} \geq h$. The maximum, and the number and share of heat days, are

$$\texttt{max}_{it} = \max_{d \in \mathcal{O}_{it}} T_{id}, \tag{11}$$

$$\texttt{heatdays}_{it} = \sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{T_{id} \geq h\}, \tag{12}$$

$$\texttt{pct_heatdays}_{it} = \frac{\texttt{heatdays}_{it}}{N_{it}}. \tag{13}$$

Let a be the lower bound supplied in `gdd_lo()` and b the upper bound supplied in `gdd_hi()`, with $b > a$. Growing degree days count, for each observed day, the degrees of temperature between a and b: 0 when $T_{id} \leq a$, $T_{id} - a$ when $a < T_{id} < b$, and $b - a$ when $T_{id} \geq b$. Daily growing degree days are thus truncated below at zero and above at $b - a$, and GDD are accumulated over the observed days as:

$$\texttt{gdd}_{it} = \sum_{d \in \mathcal{O}_{it}} \min\left\{\max\left(T_{id} - a, 0\right), b - a\right\}. \tag{14}$$

Killing degree days (KDD) measure exposure to extreme heat. Let k be the KDD base: the value supplied in `kdd_base()` or, when that option is not specified, the heat threshold h. Each observed day contributes the number of degrees by which its temperature exceeds k, and nothing when its temperature is at or below k:

$$\texttt{kdd}_{it} = \sum_{d \in \mathcal{O}_{it}} \max\left(T_{id} - k, 0\right). \tag{15}$$

Unlike GDD, KDD has no upper bound, so a few very hot days can add many degree days. The heat spells are defined in section 4.4.

GDD bins provide a nonlinear degree-day specification in the spirit of Deschênes and Greenstone (2007). When `gdd_bin()` is specified, the categorical variable `gddcat` identifies the fixed-width interval containing the accumulated GDD total of that window, $G_{it} = \texttt{gdd}_{it}$. Let ω be the interval width supplied in `gdd_bin()`, and let the cutpoints be

$$\tau_j = \tau_0 + j\omega, \qquad j = 0, 1, \ldots, Q, \tag{16}$$

where τ_0 is the value supplied in `gdd_binlo()` (default 0). When `gdd_binhi()` is specified, τ_Q is its value, and $Q = (\tau_Q - \tau_0)/\omega$ must be a whole number. Otherwise, Q is the smallest positive integer for which τ_Q exceeds the largest GDD total of all observations and windows. A bottom category, for totals below τ_0, exists only when at least one GDD total of an observation and window lies below τ_0; let β be 1 in that case and 0 otherwise. The categories are numbered consecutively from 1:

$$\texttt{gddcat}_{it} = \begin{cases} 1 & \text{if } G_{it} < \tau_0, \\ \beta + j & \text{if } \tau_{j-1} \le G_{it} < \tau_j, \quad j = 1, \ldots, Q, \\ \beta + Q + 1 & \text{if } G_{it} \ge \tau_Q. \end{cases} \tag{17}$$

The last, top-coded category arises only with `gdd_binhi()`, because without it no total reaches τ_Q. The categories are pooled across observations and windows, so a value refers to the same interval in every window, and there can be at most 100 of them. The value labels of `gddcat` identify the intervals, such as `GDD [500,600)`, so that `tabulate gddcat` displays the interval boundaries and factor-variable notation, such as `i.gddcat` (or `i.gddcat_`t in wide output), creates an indicator for each category in regressions. The command is unit agnostic, so `gdd_bin()`, `gdd_binlo()`, and `gdd_binhi()` must be expressed in the units of the generated GDD variable.

Daily temperature bins summarize the distribution of observed daily temperatures across common absolute temperature intervals in the spirit of Schlenker and Roberts (2009). These variables count days in each interval and are created when `tmp_bin()` is specified. For $J =$ `tmp_bin()` with $J \ge 3$, let $\ell =$ `tmp_binlo()`, $h =$ `tmp_binhi()`, and $w = (h - \ell)/(J - 2)$. The bin counts are

$$\texttt{tmpbin}_{it}^{(j)} = \begin{cases} \displaystyle\sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{T_{id} < \ell\}, & j = 1, \\ \displaystyle\sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{\ell + (j-2)w \le T_{id} < \ell + (j-1)w\}, & 2 \le j \le J-1, \\ \displaystyle\sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{T_{id} \ge h\}, & j = J. \end{cases} \tag{18}$$

Interior intervals include their lower boundary and exclude their upper boundary. The lower tail is strictly below ℓ and the upper tail is at or above h. Missing temperatures

are not counted in any bin. Bin values are missing when the window fails `minobs()`; otherwise their sum equals N_{it}.[3]

4.4 Dry and heat spells

A spell is a run of consecutive spell days: dry days ($R_{id} < c$) for rainfall and heat days ($T_{id} \geq h$) for temperature. The other observed days are break days: rainy days ($R_{id} \geq c$) for rainfall and cooler days ($T_{id} < h$) for temperature. A missing day is neither, and it ends a run just as a break day does. Formally, a run is a set of consecutive dates, $S = \{u, u+1, \ldots, v\} \subseteq \mathcal{W}_{it}$, all of which are spell days, such that neither $u-1$ nor $v+1$ is a spell day of the window; its length is $|S| = v - u + 1$ days. Let $\mathcal{S}_{it}$ be the set of runs of the window and $\mathcal{B}_{it}$ its set of break days.

The command reports three spells for each window: `dry_start`, `dry`, and `dry_end` for rainfall and `heat_start`, `heat`, and `heat_end` for temperature. We write their values as $start_{it}$, mid_{it}, and end_{it}. The leading spell, $start_{it}$, is the run that begins on the first observed day, $\min \mathcal{O}_{it}$, and the trailing spell, end_{it}, is the run that ends on the last observed day, $\max \mathcal{O}_{it}$:

$$start_{it} = \sum_{S \in \mathcal{S}_{it}} |S| \, \mathbf{1}\{\min S = \min \mathcal{O}_{it}\}, \tag{19}$$

$$end_{it} = \sum_{S \in \mathcal{S}_{it}} |S| \, \mathbf{1}\{\max S = \max \mathcal{O}_{it}\}. \tag{20}$$

At most one run meets each condition, so each sum is the length of that run, and it is 0 when the first or last observed day is a break day.[4]

The mid-window spell, mid_{it}, is the longest run that is neither the leading nor the trailing spell when the window has at least two break days, and the longest run anywhere in the window when it has one break day or none:

$$mid_{it} = \begin{cases} \max\{|S| : S \in \mathcal{S}_{it},\ \min S \neq \min \mathcal{O}_{it},\ \max S \neq \max \mathcal{O}_{it}\} & \text{if } |\mathcal{B}_{it}| \geq 2, \\ \max\{|S| : S \in \mathcal{S}_{it}\} & \text{if } |\mathcal{B}_{it}| \leq 1, \end{cases} \tag{21}$$

where the maximum of an empty set is 0.

An illustration of the $|\mathcal{B}_{it}| \geq 2$ case is a `heat_threshold(30)` with daily temperatures $\mathcal{W}_{it} = (35, ., 35, 35, 20, 20)$. This gives `heat_start` $= 1$, `heat` $= 2$, and

3. Two special cases apply for small J. With `tmp_bin(1)`, the single bin counts all N_{it} observed days. With `tmp_bin(2)`, the command splits at the midpoint $m = (\ell + h)/2$: the first bin counts days with $T_{id} < m$ and the second counts days with $T_{id} \geq m$.
4. Missing days before the first and after the last observed day are skipped, whereas a missing day after the first observed day ends the leading spell, and one before the last observed day ends the trailing spell.

`heat_end` = 0: the missing second day ends the leading spell, the run on days 3 and 4 is neither the leading nor the trailing spell, and the last day is a break day. Using the same heat threshold, an illustration of the $|\mathcal{B}_{it}| \leq 1$ with daily temperatures $\mathcal{W}_{it} = (35, 32, 20, 35, 35, 32)$. This gives `heat_start` = 2, `heat` = 3, and `heat_end` = 3: the cool third day ends the leading spell and the trailing spell starts the next day. Because there is only one break day, the mid-window spell is the longest spell anywhere in the window, which here is the trailing spell. A window without break days, such as a window with no cool days, therefore has a mid-window spell as long as its longest run, not 0. The same logic applies equally to dry spells.

4.5 User-defined daily bins

The `cuts()` option creates daily bin counts for either rainfall or temperature using user-specified cutpoints. Let $c_1 < \cdots < c_K$ be those cutpoints, and write X_{id} for the series being processed. The $K + 1$ counts are

$$\texttt{cut}_{it}^{(j)} = \begin{cases} \displaystyle\sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{X_{id} < c_1\}, & j = 1, \\ \displaystyle\sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{c_{j-1} \leq X_{id} < c_j\}, & 2 \leq j \leq K, \\ \displaystyle\sum_{d \in \mathcal{O}_{it}} \mathbf{1}\{X_{id} \geq c_K\}, & j = K + 1. \end{cases} \tag{22}$$

These variables are named `cut01`, `cut02`, and so on, with a starting-year suffix in calendar-window wide output. They count observed days rather than shares, and their sum is N_{it} when the window meets `minobs()`. The cutpoints need not be equally spaced. This option is an alternative to `tmp_bin()`, not an additional set of temperature bins in the same command.

4.6 Long-run reference periods

Several variables generated by `wxsum` describe weather shocks as deviations from a location-specific reference period. Let w_{it} denote one of the window statistics for which these measures are generated: `mean_mo`, `total`, `raindays`, `norain`, `pct_raindays`, or `dry` for rainfall; `gdd`, `kdd`, `heatdays`, `pct_heatdays`, or `heat` for temperature.

Using either `lr_years()` or `baeline()`, the reference period of a window that starts in year t is a set of years, $\mathcal{Y}_t$. Let $\mathcal{Y}_{it} \subseteq \mathcal{Y}_t$ be the set of years with a nonmissing reference value and let $r_{it} = |\mathcal{Y}_{it}|$ be their number.[5] The long-run mean and standard deviation are those of these r_{it} values,

5. A reference value is missing when its window would have missing statistics: when the window has fewer than `minobs()` observed days or, unless `partial` is specified for a unit-specific window, when it does not lie within the date range of the data.

$$\bar{w}_{it} = \frac{1}{r_{it}} \sum_{t' \in \mathcal{Y}_{it}} w_{it'}, \tag{23}$$

$$s_{it} = \left\{ \frac{1}{r_{it} - 1} \sum_{t' \in \mathcal{Y}_{it}} (w_{it'} - \bar{w}_{it})^2 \right\}^{1/2}, \tag{24}$$

and the deviation and corresponding z-score are

$$\texttt{dev_}w_{it} = w_{it} - \bar{w}_{it}, \tag{25}$$

$$\texttt{z_}w_{it} = \frac{w_{it} - \bar{w}_{it}}{s_{it}}. \tag{26}$$

Both require that w_{it} be nonmissing and that the observation have at least ρ reference values, $r_{it} \geq \rho$, where ρ is the value supplied in `ref_min()`. By default, ρ equals the number of reference years, $|\mathcal{Y}_t|$, so every reference year must have a value. The z-score is also missing when the reference values do not vary, so that $s_{it} = 0$. With the default `lr_years(10)`, for example, the first windows of the data have no deviations, because fewer than 10 earlier windows lie within the date range of the data.

5 Examples

The package includes small rainfall and temperature datasets that can be used to check installation and illustrate the syntax. These files are also useful as templates for checking the required wide layout and variable-naming conventions. In addition to those illustrative data sets, `wxsum` comes with simulated data and a do-file with a set of 12 worked examples.

5.1 Rainfall summaries

Suppose that the daily rainfall variables in `rain.dta` are named with the prefix `rf_`. The following commands calculate seasonal rainfall statistics for a May 15 through October 15 growing season and save the resulting dataset:

```
. use rain.dta, clear
 . wxsum rf_, type(rain) ini_month(05) fin_month(10) ini_day(15) ///
 >     fin_day(15) rain_threshold(1) keep(hhid) ///
 >     save(rainfall_stats.dta)
```

The output dataset contains one observation for each input location. For each season observed in the daily rainfall variables, `wxsum` adds the rainfall statistics listed in Table 2. If the user also needs location identifiers or other merge variables, these should be supplied in `keep()`. The resulting file can then be merged with a socioeconomic

dataset using the retained identifier variables and the season-specific weather variables generated.

5.2 Temperature summaries

Suppose that the daily temperature variables in `temp.dta` are named with the prefix `tmp_`. The following commands calculate seasonal temperature statistics for a November through February growing season. As the final month precedes the initial month, the command treats the exposure window as spanning calendar years and assigns the season to the year in which it begins:

```
. use temp.dta, clear
 . wxsum tmp_, type(temp) ini_month(11) fin_month(02) gdd_lo(8)  ///
 >     gdd_hi(32) kdd_base(32) keep(hhid)
```

This command generates the temperature summaries listed in Table 3. The values `gdd_lo(8)`, `gdd_hi(32)`, and `kdd_base(32)` are illustrative; they should be chosen to match the units of the underlying temperature data and the biological or behavioral process being studied.

5.3 GDD and temperature bins

To create Deschênes–Greenstone-style GDD bins, the user specifies `gdd_bin()` with a bin width in the units of the generated GDD variable:

```
. use temp.dta, clear
 . wxsum tmp_, type(temp) ini_month(04) ini_day(01) fin_month(09) ///
 >     fin_day(15) gdd_lo(18) gdd_hi(30) kdd_base(32) gdd_bin(100) ///
 >     gdd_binlo(50) gdd_binhi(950) keep(hhid)
```

Here `gdd_bin(100)` means the bin width will be 100 degree-days in the units of the generated GDD variable. In this example the user set `gdd_binhi()` and `gdd_binlo()` though these are not required. In the absence of these options, the default lower endpoint is 0 while the command determines the upper endpoint from the pooled empirical maximum of GDD. This example assumes Celsius temperatures and Celsius degree-days. The command does not infer or convert units.

To create Schlenker–Roberts-style temperature bins, the user specifies `tmp_bin()` along with `tmp_binlo()` and `tmp_binhi()`:

```
. use temp.dta, clear
 . wxsum tmp_, type(temp) ini_month(04) ini_day(01) fin_month(09) ///
 >     fin_day(15) gdd_lo(18) gdd_hi(30) kdd_base(32) tmp_bin(15)  ///
 >     tmp_binlo(0) tmp_binhi(39) keep(hhid)
```

Here `tmp_bin(15)`, `tmp_binlo(0)`, and `tmp_binhi(39)` create 15 bin-count variables per season: a lower tail counting days below 0, 13 equal-width interior intervals over $[0, 39)$, and an upper tail counting days at or above 39. This is a Celsius-style example; the command does not infer or convert units.

5.4 Changing the long-run benchmark

The default long-run reference period is 10 prior years. Researchers working with longer weather histories may prefer a longer benchmark, whereas applications with short panels may require a shorter one. The following command uses the prior 20 years as the rolling reference period:

```
. use rain.dta, clear
 . wxsum rf_, type(rain) ini_month(05) fin_month(10) lr_years(20) ///
 >     keep(hhid)
```

Changing `lr_years()` affects only the rolling deviations and z-scores. It does not affect the seasonal means, medians, variances, standard deviations, skewness measures, totals, counts, shares, dry-spell measures, GDD categories, or daily-temperature bin counts.

5.5 Long output

To produce long-shaped output, the user specifies `shape(long)`:

```
. use rain.dta, clear
 . wxsum rf_, type(rain) ini_month(05) fin_month(10) ini_day(15) ///
 >     fin_day(15) rain_threshold(1) keep(hhid) ///
 >     shape(long)
```

The output has one row per `id`-year. The time variable is named `year`. Variables are named `mean`, `median`, and so on, rather than `mean_`*YYYY*, `median_`*YYYY*, etc.

6 Conclusions

`wxsum` reduces the amount of custom code needed to transform daily weather data into seasonal measures commonly used in socioeconomic analysis. It standardizes the construction of summary statistics, threshold-based measures, rainy-day counts, dry spells, GDD bins, daily temperature bins, and rolling weather-shock measures. The command handles cross-year seasons, missing data, and large datasets without requiring users to reshape or preprocess their data beyond the standard wide format. An implementation for R has also been developed, extending the same core functionality to users who work outside Stata or who integrate weather processing into R-based data pipelines.

While standardizing calculations, `wxsum` leaves substantive choices with the researcher by allowing users to customize the season definition, the thresholds for growing and killing degree days, GDD bin widths and endpoints, daily temperature bin counts and endpoints, the rainfall threshold, the length of the long-run window used to compute deviations and z-scores, and the shape of the output data frame. This division of labor is useful: the command reduces the likelihood of programming errors in repetitive weather-processing tasks, but it does not substitute for careful research design.

7 References

Auffhammer, M., and A. Aroonruengsawat. 2011. Simulating the Impacts of Climate Change, Prices and Population on California's Residential Electricity Consumption. *Climatic Change* 109(Suppl 1): S191–S210.

Auffhammer, M., S. Hsiang, W. Schlenker, and A. Sobel. 2013. Using Weather Data and Climate Model Output in Economic Analyses of Climate Change. *Review of Environmental Economics and Policy* 7(2): 181–198.

Azzarri, C., and S. Signorelli. 2020. Climate and Poverty in Africa South of the Sahara. *World Development* 125: 104691.

Benami, E., J. D. Michler, A. Josephson, M. Cecil, and G. Maskell. 2026. The Uses (and Misuses) of Earth Observation Data on Weather and Vegetation. In *Geospatial Impact Evaluation in Practice*, ed. A. BenYishay and K. Singh. Taylor & Francis.

Bosilovich, M., R. Lucchesi, and M. Suarez. 2016. MERRA-2: File Specification. GMAO Office Note No. 9 (Version 1.1). http://gmao.gsfc.nasa.gov/pubs/office_notes.

Chen, M., W. Shi, P. Xie, V. B. Silva, V. E. Kousky, R. W. Higgins, and J. E. Janowiak. 2008. Assessing objective techniques for gauge-based analyses of global daily precipitation. *Journal of Geophysical Research* 113: D04110.

Deschênes, O., and M. Greenstone. 2007. The Economic Impacts of Climate Change: Evidence from Agricultural Output and Random Fluctuations in Weather. *American Economic Review* 97(1): 354–85.

Dinku, T. 2020. Remote sensing of African rainfall. *Oxford Research Encyclopedia of Climate Science* 30 July.

Doyle, M.-A., and B. Leckning. 2025. The Relationship between Prenatal Heat Exposure and Birth Outcomes: How Much Does the Heat Metric Matter? *PLOS ONE* 20(9): e0330498.

Funk, C., P. Peterson, M. Landsfeld, D. Pedreros, J. Verdin, S. Shukla, G. Husak, J. Rowland, L. Harrison, A. Hoell, and J. Michaelsen. 2015. The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes. *Scientific Data* 2: 150066.

Gortan, M., L. Testa, G. Fagiolo, and F. Lamperti. 2024. A Unified Dataset for Pre-Processed Climate Indicators Weighted by Gridded Economic Activity. *Scientific Data* 11: 533.

Hendrix, C. S., and I. Salehyan. 2012. Climate Change, Rainfall, and Social Conflict in Africa. *Journal of Peace Research* 49(1): 35–50.

Hersbach, H., B. Bell, P. Berrisford, G. Biavati, A. Horányi, J. Muñoz Sabater, J. Nicolas, C. Peubey, R. Radu, I. Rozum, A. Schepers, D.and Simmons, C. Soci, D. Dee, and J.-N. Thépaut. 2018. ERA5 hourly data on single levels from 1940

to present. Copernicus Climate Change Service (C3S) Climate Data Store (CDS). 10.24381/cds.adbb2d47.

Josephson, A., J. D. Michler, T. Kilic, and S. Murray. 2026. The mismeasure of weather: Using earth observation data for estimation of socioeconomic outcomes. *Journal of Development Economics* 178: 103553.

Marcos-Garcia, P., C. Carmona-Moreno, and M. Pastori. 2024. Intra-Growing Season Dry–Wet Spell Pattern Is a Pivotal Driver of Maize Yield Variability in Sub-Saharan Africa. *Nature Food* 5: 775–786.

Mellon, J. 2025. Rain, Rain, Go Away: 194 Potential Exclusion-Restriction Violations for Studies Using Weather as an Instrumental Variable. *American Journal of Political Science* 69(3): 881–898.

Michler, J. D., K. Douglas, and A. Josephson. 2026a. Mining Meaning: Measurement Error in AI-Assisted Literature Reviews. arXiv: 2609.27686.

Michler, J. D., A. Josephson, E. Benami, P. Behrer, M. J. Cecil, S. Gourlay, R. Heilmayr, E. Kirchner, G. Maskell, and K. Singh. 2026b. Treatment Geometry and Causal Identification with Earth Observation Data. In *Geospatial Impact Evaluation in Practice*, ed. A. BenYishay and K. Singh. Taylor & Francis.

Michler, J. D., A. Josephson, T. Kilic, and S. Murray. 2022. Privacy protection, measurement error, and the integration of remote sensing and socioeconomic survey data. *Journal of Development Economics* 158: 102927.

Novella, N. S., and W. M. Thiaw. 2013. African Rainfall Climatology Version 2 for Famine Early Warning Systems. *Journal of Applied Meteorology and Climatology* 52(3): 588–606.

Park, R. J., J. Goodman, M. Hurwitz, and J. Smith. 2020. Heat and Learning. *American Economic Journal: Economic Policy* 12(2): 306–339.

Proctor, J., T. Carleton, and S. Sum. 2023. Parameter Recovery Using Remotely Sensed Variables. NBER Working Paper 30861.

Sarsons, H. 2015. Rainfall and Conflict: A Cautionary Tale. *Journal of Development Economics* 115: 62–72.

Schlenker, W., and M. J. Roberts. 2009. Nonlinear temperature effects indicate severe damages to U.S. crop yields under climate change. *Proceedings of the National Academy of Sciences* 106(37): 15594–15598.

Somanathan, E., R. Somanathan, A. Sudarshan, and M. Tewari. 2021. The Impact of Temperature on Productivity and Labor Supply: Evidence from Indian Manufacturing. *Journal of Political Economy* 129(6): 1797–1827.

Tarnavsky, E., D. Grimes, R. Maidment, E. Black, R. P. Allan, M. Stringer, R. Chadwick, and F. Kayitakire. 2014. Extension of the TAMSAT Satellite-Based Rainfall Monitoring over Africa and from 1983 to Present. *Journal of Applied Meteorology and Climatology* 53(12): 2805–22.

Wuepper, D., W. A. Oluoch, and H. Hadi. 2025. Satellite Data in Agricultural and Environmental Economics: Theory and Practice. *Agricultural Economics* 56(3): 493–511.

Acknowledgments

We thank Brian McGreal for help in the early debugging of the `wxsum` command and Emanuele Clemente for final testing. Generative artificial intelligence tools were used during the final debugging and manuscript preparation process. Specifically, Anthropic's Claude Opus 4.6 Thinking, OpenAI's ChatGPT 5.5 Thinking, and Google Lab's Jules based on Gemini 3.1 Pro were used in final debugging, testing, and validation. The same Claude and GPT models were used to edit and format the article's text. The authors wrote, reviewed, and are responsible for the final content of the article and software.

About the authors

Jeffrey D. Michler is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Anna Josephson is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Aleksandr Michuda is a Visiting Assistant Professor at the Department of Economics at Swarthmore College.

Oscar Barriga-Cabanillas is an Economist at the World Bank.

Jeffrey C. Oliver is a Data Science Specialist in the University of Arizona Libraries.